

Part 01 – Theoretical Questions

1. Why is it better to code against an interface rather than a concrete class?

It reduces coupling and makes the code more flexible by allowing different implementations.

2. When should you prefer an abstract class over an interface?

When related classes need shared state, common implementation, or common behavior.

3. How does implementing `Comparable` improve flexibility in sorting?

It defines a default comparison between objects, allowing sorting methods such as `Array.Sort()` to sort them automatically.

4. What is the primary purpose of a copy constructor in C#?

To create a new object by copying the values of an existing object.

5. How does explicit interface implementation help in resolving naming conflicts?

It allows a class to provide a separate implementation for an interface member and resolve naming conflicts.

6. What is the key difference between encapsulation in structs and classes?

Both support encapsulation, but a struct is a value type while a class is a reference type.

7. What is abstraction as a guideline, and what is its relation with encapsulation?

Abstraction means focusing on what an object does while hiding unnecessary implementation details. Encapsulation controls access to the object's internal data and is one way to achieve abstraction.

8. How do default interface implementations affect backward compatibility in C#?

They allow new interface members to have default implementations without requiring existing implementing classes to immediately implement them.

9. How does constructor overloading improve class usability?

It allows objects to be created using different sets of parameters, making the class more flexible and easier to use.

Part 02 – Theoretical Questions

10. What do we mean by coding against an interface rather than a class?

It means depending on an interface or abstraction instead of a specific concrete implementation.

11. What do we mean by coding against abstraction, not concreteness?

It means that code should depend on general abstractions, such as interfaces or abstract classes, rather than specific concrete classes.

12. What is abstraction as a guideline?

Abstraction as a guideline means designing code around what an object does rather than how it does it.

13. How can abstraction be implemented through what we have studied?

Abstraction can mainly be implemented using interfaces and abstract classes.